

AI Native SOP 可视化看板前端形态与 Artifact 交接机制调研报告

基于 Office Map、Role-based Workspace、Artifact Panel 与 Artifact Handoff Layer 的界面模式研究

维度	内容
报告定位	前端形态调研报告 + Artifact 交接机制调研报告。
核心问题	不仅回答界面怎么展示，还回答 PRD、代码变更、测试报告如何从一个岗位可靠地传到下一个岗位。
核心假设	Workspace 内部保留 Context；Workspace 之间只流转经过整理、审核和标准化的 Artifact。
推荐方向	Office Map + Role-based Workspace + Artifact Panel + Lightweight Artifact Handoff。
MVP 范围	先做 PM -> Dev -> QA 单链路，重点展示 Submit、Inbox、Status Update 和 Handoff 动效。

一句话结论

新版方案应把 Artifact Handoff 从“好看的交接动画”升级为“可被客户理解的产品机制”：产物有来源、有版本、有接收状态、有审核动作，也能在 Office Map 中被看见。

0. Executive Summary / 核心结论

本次调研围绕 AI Native SOP 看板的前端形态与 Artifact 交接机制展开。与上一版只关注 Office Map、Workspace、Artifact Panel 三层形态不同，新版报告进一步补上了岗位之间的可靠传递机制：不同 Workspace 不直接共享完整 Context，而是通过 Artifact Handoff Layer 传递经过整理、审核和权限控制的标准化 Artifact。

- 顶层应采用 Office Map，让客户快速理解每个岗位都有员工与 AI 在工作。
- 中层应采用 Role-based Workspace，承载不同岗位的真实工作过程和岗位专属工具。
- 底层应采用差异化 Artifact Panel，分别展示 PRD、代码变更、测试报告等标准化产物。
- 岗位之间不应传递完整聊天 Context，而应通过 Artifact Handoff Layer 完成提交、接收、通知、审核和状态更新。
- 第一版建议采用轻游戏化 Office Map + 专业 Workspace + 差异化 Artifact Panel + Artifact Inbox / Handoff 机制的混合形态。

推荐产品方向

第一版采用 Office Map + Role-based Workspace + Artifact Panel + Lightweight Artifact Handoff 的四部分结构。

1. 项目背景与问题定义

1.1 项目背景

在 AI 原生 workflows 中，AI 不再只是一个聊天工具，而是嵌入产品、开发、测试等岗位流程中的协作者。但对客户来说，AI 的工作过程通常不可见，客户只能看到最终结果，难以理解 AI 在不同岗位中如何创造价值，也难以判断这些产物能否可靠进入企业流程。

因此，本项目要解决的是两个层面的问题：第一，前端如何把岗位、人和 AI 的协作讲清楚；第二，产物如何作为标准化对象从一个岗位可靠交接给下一个岗位。

1.2 当前问题

问题	说明	对前端 / 机制的要求
AI 工作不可见	客户看不到 AI 如何参与 PM、Dev、	Office Map 和 Workspace 需要展示 AI 的状

workflows表达抽象	QA 的具体工作。 普通流程图难以表达“人 + AI”在岗位内协同。	态、任务和产物。 用办公室和岗位空间替代单纯节点流转。
产物缺乏可信度	只展示“AI 在工作”不够，还需要展示标准化产物。	Artifact Panel 必须岗位差异化、可审查。
上下游交接不清晰	PRD、代码变更、测试报告如何传递需要机制支撑。	需要 Artifact Store、Inbox、Submit/Accept 状态。
Context 边界模糊	直接共享聊天记录会造成信息冗余、权限风险和下游理解负担。	Workspace 内部处理 Context，外部只流转 Artifact。

2. 核心产品假设

2.1 我们设计的不是 Workflow，而是 Workspace

传统 Workflow 强调流程节点如何流转，本项目更关注每个岗位内部的人机协作空间。一个岗位不是流程图上的一个圆点，而是一个可被进入、可被观察、可输出产物的 Workspace。

核心模型

Workspace（人 + AI Agent） -> Artifact -> Workspace（人 + AI Agent） -> Artifact -> Workspace（人 + AI Agent）

2.2 Workspace 内部处理 Context，Workspace 之间只流转 Artifact

每个 Workspace 内部可以包含聊天、Prompt、工具调用、文件编辑、代码生成、测试执行等复杂上下文，但这些内部 Context 不直接传递给下游。下游只消费经过整理、审核和标准化后的 Artifact，例如 PRD、Code Change、Test Report。

这个原则的意义在于：既降低跨岗位理解成本，也减少权限和噪音风险；同时让每个阶段的输出可以被企业审查、归档、复用和追踪。

2.3 前端形态假设

层级	作用	关键表达
Office Map	让客户快速理解每个岗位都有人和 AI 在工作。	办公室、工位、角色、状态、关键 Artifact 和交接路径。

Role-based Workspace	承载岗位内部真实工作过程。	PM / Dev / QA 各自拥有不同的工具、面板和 AI 搭档。
Artifact Panel	展示标准化产物，保证 B 端可信度。	PRD、Code Change、Test Report 的专业面板。
Artifact Handoff Layer	负责岗位之间的产物流转、通知、权限和审计。	Submit、Store、Inbox、Accept / Reject、版本、日志。

3. 总体产品架构假设

3.1 四部分结构

模块	产品角色	第一版展示重点
Office Map	顶层叙事界面，让客户看到岗位、人、AI 和流程状态。	Product Office、Dev Office、QA Lab 以及 Artifact 交接路径。
Role-based Workspace	岗位专属工作区，员工与 AI 在这里完成真实工作。	PM: PRD Builder；Dev: Code/Diff Preview；QA: Test Console。
Artifact Panel	标准化产物展示层。	PRD、Code Change、Test Report 三种差异化面板。
Artifact Handoff Layer	产物流转机制，负责提交、接收、权限、通知和审计。	Submit -> Inbox -> Status Update -> Handoff Animation。

3.2 PM -> Dev -> QA 示例流程

- PM Workspace 中，人类 PM 与 PM Agent 共同整理需求并生成 PRD Artifact。
- PRD 经过人工审核后提交到 Artifact Store。
- Dev Workspace 收到 PRD Inbox 通知，Developer 与 Coding Agent 基于 PRD 完成功能。
- Dev Workspace 输出 Code Change / Feature Artifact，包括 Diff、Commit、Build 状态和预览。
- QA Workspace 收到 Feature Artifact，Tester 与 QA Agent 生成测试用例、缺陷列表和 Test Report。
- Manager View 可以查看全局状态、阻塞点、Artifact 交接记录和关键证据。

3.3 核心原则

原则	说明
Workspace 内部保留 Context	聊天、工具调用、草稿、临时文件都属于本岗位内部，不直接传给下游。

- Workspace 之间只传 Artifact

Artifact 需要审核和版本

Handoff 需要可视化

权限按角色控制
- 下游只消费标准化产物，减少噪音和权限风险。

避免 AI 直接把不稳定结果传给下游。

交接不仅是后端事件，也要在客户演示中被看见。

员工、管理者、AI Agent 应看到不同层级的内容。

4. 调研目标与范围

4.1 调研目标

本次调研的目标不是寻找完全相同的产品，而是寻找能够分别支撑 Office Map、Workspace、Artifact Panel 和 Artifact Handoff Layer 的产品案例、界面模式和机制设计。最终报告应帮助 mentor 判断：第一版应如何在“客户理解成本”“B 端可信度”“机制完整性”和“MVP 可落地性”之间取平衡。

4.2 调研范围

调研类别	对应模块	调研目的	代表案例
虚拟办公室 / AI 员工实体化	Office Map	研究如何让岗位、人、AI 具象化。	My Virtual Office、WorkAdventure、Gather、AgentOffice
AI 工作区 / Coding Workspace	Workspace	研究人和 AI 如何在一个工作台协作。	Cursor、Claude Code、Codex、OpenHands、Replit Agent
Artifact / Report / Review 面板	Artifact Panel	研究标准化产物如何展示。	Confluence PRD、Linear Docs、GitHub PR、Allure、Playwright
Agent Coworking / Handoff 机制	Artifact Handoff Layer	研究上下游如何传递 Artifact。	Claude Agent Coworking、GitHub PR、Jira、Linear、n8n
Workflow / Pipeline 可视化	Handoff + Office Map	研究流程状态、成功失败、等待审批。	n8n、Dify Workflow、Langflow、GitHub Actions
Agent Observability	管理者 / 技术下钻	研究 AI 工作过程如何审计和追踪。	AgentPrism、ClawMetry、LangSmith、Arize Phoenix

5. Interface Pattern Benchmark / 界面模式调研

5.1 Pattern 1: Office Map / 办公室地图模式

Office Map 解决的是第一眼认知问题：让客户快速理解每个岗位都有员工和 AI 在共同工作。它不是纯观赏动画，而是业务状态地图。

案例	类型	可借鉴点	限制
My Virtual Office	开源 AI Agent 虚拟办公室	AI Agent 实体化、2D 办公室、实时状态，契合“AI 正在干活”。	仍偏开发者概念产品，需要增强企业级信息层级。
WorkAdventure	开源虚拟办公室	房间、工位、角色移动、空间交互、自托管。	偏远程办公，不直接表达 AI 产物。
Gather	商业虚拟办公室	常驻空间、角色存在感、自然协作。	更偏沟通协作，不等同于 SOP 看板。
AgentOffice	AI 像素办公室	Agent 状态动画、任务表现、团队感。	纯像素风有 B 端可信度风险。

启发：顶层采用轻游戏化 Office Map，展示 Product Office、Dev Office、QA Lab。每个办公室默认展示人类员工、AI 搭档、当前任务、当前状态和最新 Artifact。

5.2 Pattern 2: Role-based Workspace / 岗位工作空间模式

Workspace 解决的是可信执行问题：用户点击工位后，需要看到该岗位真正可工作的专业界面，而不是继续停留在办公室动画里。

案例	类型	可借鉴点	限制
Cursor	AI Coding Workspace	Chat + IDE + 文件 + Diff，适合 Dev Workspace。	主要服务开发角色，PM/QA 需要重新设计。
Claude Code	Agentic Coding Tool	代码库理解、命令执行、工具调用和子 Agent 能力。	偏工程师使用，客户演示需要降噪。
Codex	AI Coding Agent	可读、改、运行代码，并支持多种开发表面。	要避免把 Demo 变成纯开发工具展示。
OpenHands	开源软件开发 Agent 平台	Agent 执行工程任务的工作台。	技术复杂度较高。
Replit Agent	AI App Builder	对话、生成、预览、部署一体化。	更像应用生成平台，不是企业 SOP 看板。

Dify	AI Workflow / Agent 平台	Workflow、RAG、调试、发布。	偏 AI 应用搭建，不强调岗位办公室。
------	------------------------	---------------------	---------------------

启发：PM Workspace 可采用 Chat + Doc Editor + PRD Builder；Dev Workspace 可采用 IDE + AI Coding Agent + Diff + Preview；QA Workspace 可采用 Test Console + Bug List + Report Builder。

5.3 Pattern 3: Artifact Panel / 标准化产物面板模式

Artifact Panel 解决的是 B 端可信度问题：客户和管理者不仅要看到“这个岗位做完了”，还要看到该岗位产出了什么标准化业务对象。

岗位	Artifact 类型	核心字段	参考案例
PM	PRD / User Stories / Acceptance Criteria	业务目标、用户场景、功能范围、优先级、验收标准、依赖和风险。	Confluence PRD、Linear Project Docs
Dev	Code Change / Pull Request / Build	提交说明、Diff、Changed Files、Commit、Review、Checks、构建状态。	GitHub Pull Request、GitHub Actions
QA	Test Report / Bug List / Regression Result	测试范围、通过率、失败用例、截图/附件、缺陷列表、回归结论。	Allure Report、Playwright HTML Report

启发：Artifact 不能统一设计成普通文件卡片。不同岗位的 Artifact 应采用不同专业面板，但共享元信息骨架：类型、来源、版本、状态、提交人、审核人、下游接收状态和审计日志。

5.4 Pattern 4: Artifact Handoff Layer / 产物交接机制

这是新版报告最关键的新增层。它解决岗位之间如何可靠传递产物的问题：不是把完整聊天 Context 传给下游，而是通过标准化 Artifact 进行正式交接。

案例 / 机制	类型	可借鉴点	对本项目的启发
Claude Agent Coworking Workflows	文章 / 机制参考	共享环境、持久化存储、任务交接、人机协作。	可借鉴“交接产物是协作契约”的思路，但需结合企业权限和审计。
GitHub PR Review Flow	代码交接	Submit、Review、Approve、Merge。	Dev Artifact 可采用正式审核状态，而不是简单文件传递。
GitHub Protected Branches	交接控制	要求合并前获得指定数量审核。	高风险 Artifact 可要求人工审批后流转。
Jira / Linear	项目管理	状态流转、负责人变化、Issue 生命周期。	Artifact 可有 Submitted、Received、In Progress、

n8n	Workflow Automation	节点输入输出、状态流转、失败处理。	Done、Rejected 状态。 Artifact handoff 可以被表现为节点输入输出与事件触发。
-----	---------------------	-------------------	--

可提炼机制

Generate Artifact -> Human Review -> Submit Artifact -> Artifact Store -> Notify Next Workspace -> Artifact Inbox -> Accept / Reject / Request Clarification -> Start Next Workspace Task

启发：PM 生成 PRD 后，不是把聊天记录传给开发，而是把 PRD Artifact 提交到 Artifact Store。Dev Workspace 收到 New PRD Inbox 通知；Office Map 中触发 PRD 从 PM Office 递交到 Dev Office 的事件动画；Artifact Panel 中记录版本、提交人、审核人、接收状态和审计日志。

5.5 Pattern 5: Workflow / Pipeline 可视化模式

Workflow / Pipeline 可视化解决的是流程推进状态问题。Office Map 不需要展示复杂流程图，但需要让客户看到关键状态：Working、Waiting for Artifact、Reviewing、Submitted、Blocked、Done。

案例	类型	可借鉴点	对本项目的启发
n8n	Workflow Automation	节点、输入输出、执行状态。	用于理解 Artifact 如何触发下游任务。
Dify Workflow	AI Workflow	AI 节点、流程调试、输入输出。	可借鉴 AI 节点的调试和执行状态。
Langflow	Visual AI Flow	节点式 AI 编排。	可借鉴可视化连接关系。
GitHub Actions	DevOps Pipeline	Workflow Run、Job、Step、Log、成功失败状态。	QA / Build 类 Artifact 可借鉴 Job/Step/Log 层级。

5.6 Pattern 6: Agent Observability / AI 工作过程下钻

Agent Observability 解决的是过程证据问题。普通客户不需要在顶层看到完整 Trace，但管理者或技术用户需要进一步确认 AI 到底执行了哪些工具、在哪一步失败、耗时和成本如何。

案例	类型	可借鉴点	放置建议
AgentPrism	Agent Trace 可视化	Plan、Action、Tool Call、Retry 时间线。	放在 Workspace 或 Artifact 的高级下钻中。
ClawMetry	Agent Observability Dashboard	Agent 状态、Token、成本、运行监控。	用于管理者视角。

LangSmith	LLM Observability	Trace、Latency、Cost、Debug。	用于证明 AI 过程可追溯。
Arize Phoenix	AI Observability / Evaluation	Trace Viewer、Tool Use、Retrieval。	作为证据层扩展。

启发：Agent Trace 不应放在顶层 Office Map。更适合放在 Workspace 或 Artifact Panel 的高级下钻中，作为过程证据。

6. 重点案例分析

案例	对应模块	核心价值	可借鉴设计 / 机制	风险 / 不适合点
My Virtual Office	Office Map	让 AI Agent 有可见位置和状态。	AI 员工常驻、办公室地图、实时状态。	需要企业级信息设计。
Cursor / Claude Code / Codex	Workspace	展示人和 AI 在专业工作台协作。	Chat + Context + Tool + Diff / Preview。	不应让客户演示陷入开发工具细节。
Confluence / Linear Docs	Artifact Panel	将 PRD、Spec、状态更新组织成结构化文档。	PM Artifact 的专业结构和项目关联。	需要避免文档过重。
GitHub Pull Request	Artifact + Handoff	把代码变更、讨论、审查和合并放在一个对象里。	Submit、Review、Approve、Merge 状态。	需简化技术术语。
GitHub Actions / Allure / Playwright	QA Artifact + Pipeline	用测试结果、失败详情、日志支撑质量判断。	Job/Step/Log、Pass/Fail、附件和趋势。	测试细节不宜挤占演示节奏。
Jira / Linear Workflow	Handoff Status	用状态和负责人表达任务生命周期。	Submitted、Received、In Progress、Done、Rejected。	不是前端主叙事，只作机制参考。
AgentPrism / LangSmith / Phoenix	Observability	将 AI 执行过程变成可追踪证据。	Trace、Tool Call、Cost、Latency、Retry。	普通客户首屏不展示。

7. 对比矩阵

7.1 前端模块与交接机制映射

模块	目标	可借鉴案例	关键模式
----	----	-------	------

Office Map	快速理解人机协作。	My Virtual Office、WorkAdventure、Gather	空间化、角色化、状态气泡、事件动效。
Workspace	承载真实工作。	Cursor、Claude Code、Codex、OpenHands	Chat + Editor + Tool + Preview。
Artifact Panel	展示可信产物。	Confluence、Linear、GitHub PR、Allure	PRD、Diff、Report、版本、审核。
Artifact Handoff Layer	解决上下游传递。	Claude Coworking、n8n、GitHub PR、Jira	Artifact Store、Inbox、Submit、Review。
Pipeline Status	展示流程推进。	GitHub Actions、n8n、Dify	Running、Failed、Done、Blocked。
Observability	展示 AI 过程证据。	AgentPrism、ClawMetry、LangSmith	Trace、Tool Call、Cost、Audit。

7.2 不同方案对比

方案	优点	风险	适合场景	建议
纯 Workflow Dashboard	清晰、专业、易落地。	客户感知弱，缺少记忆点。	内部管理。	不作为第一推荐。
纯游戏化办公室	生动、容易理解。	B 端可信度不足。	概念 Demo。	只借鉴隐喻和动效。
纯专业 Workspace	真实可用。	顶层表达不直观。	员工实际工作。	作为中层能力表达。
Office Map + Workspace + Artifact Panel	兼顾理解、使用、可信度。	仍未解释上下游交接机制。	客户演示 + 内部管理。	作为基础结构。
加入 Artifact Handoff Layer	交接机制完整，更像真实产品。	复杂度上升。	MVP 后续增强 / 正式产品化。	推荐采用轻量版。

8. 设计模式总结

- Workspace as Office：每个岗位被表达为办公室或工位。
- Human + AI as Pair：每个岗位都是人类员工 + AI 搭档。
- Artifact as Handoff Object：PRD、代码变更、测试报告是岗位之间的交接对象。
- Artifact Inbox：下游 Workspace 通过 Inbox 接收上游产物。
- Submit / Review / Accept：Artifact 交接包含正式状态，不是简单传文件。
- Status as Motion：用角色动作、状态气泡、路径动画表达流程推进。
- Drill-down from Narrative to Evidence：顶层讲故事，下钻看 Workspace、Artifact 和 Trace 证据。

- Auditability by Design：每个 Artifact 记录生成者、审核者、版本、时间、接收状态和修改记录。

9. 推荐产品方向

9.1 推荐方向

最终建议
第一版采用 Office Map + Role-based Workspace + Artifact Panel + Lightweight Artifact Handoff 的四部分结构。

9.2 推荐原因

原因	说明
客户理解成本低	Office Map 让客户快速理解岗位正在工作。
AI 价值可见	人和 AI 搭档同时出现在岗位中。
B 端可信度高	Artifact Panel 展示真实业务产物，而不是泛泛的 AI 输出。
交接逻辑完整	Artifact Handoff Layer 解释上下游如何可靠传递。
可逐步落地	MVP 可以先做 PM -> Dev -> QA 三个岗位。
有扩展空间	后续可加入权限、审计、Webhook、File Lock、Agent Trace。

10. MVP 范围建议

10.1 第一版只做 PM -> Dev -> QA

环节	人机协作	输出 Artifact	交接动作
Product Office	PM + PM Agent 整理需求。	PRD Artifact	Submit PRD 到 Artifact Store。
Dev Office	Developer + Coding Agent 根据 PRD 开发。	Code Change / Feature Artifact	Dev Inbox 接收 PRD，完成后 Submit Feature。
QA Lab	Tester + QA Agent 生成测试用例并执行。	Test Report Artifact	QA Inbox 接收 Feature，输出测试报告。

10.2 MVP 页面范围

页面	内容
Office Map 首页	三个办公室、角色状态、Artifact 交接路径。
PM Workspace	Chat + PRD Builder + Submit PRD。
Dev Workspace	PRD Inbox + Coding Workspace + Code Artifact。
QA Workspace	Feature Inbox + Test Console + Test Report。
Artifact Panel	PRD / Code Change / Test Report 三种面板。
Manager View	全局状态、阻塞点、Artifact 列表。

10.3 MVP Handoff 机制

轻量版 Handoff

Submit Artifact -> 下游 Inbox 出现新 Artifact -> Office Map 播放交接动画 -> 状态更新为 Received / In Progress / Done

后续再加入权限控制、版本管理、审计日志、Webhook、File Lock 和 Agent Trace。第一版重点是让客户理解“产物怎么可靠传下去”，不是先实现完整协同系统。

11. 风险与注意事项

风险	表现	应对策略
游戏化过度	办公室地图过于像小游戏，削弱 B 端可信度。	采用轻游戏化、企业级视觉风格。
Context 共享风险	Workspace 之间直接共享全部上下文，造成信息冗余、权限混乱和下游理解负担。	坚持 Workspace 内部处理 Context，Workspace 之间只流转 Artifact。
Artifact 同质化	所有 Artifact 都是普通文件卡，无法体现岗位差异。	PRD、Code Change、Test Report 分别采用不同专业面板。
交接机制复杂度风险	第一版就做完整权限、Webhook、审计、文件锁，成本过高。	MVP 先实现 Submit -> Inbox -> Status Update -> Animation。
Trace 暴露过多	把技术执行细节放在客户首屏，增加理解负担。	Trace 只作为高级下钻，面向管理者或技术用户。

12. 下一步计划

步骤	任务	产出
Step 1	按六类建立案例池：Office Map、Workspace、Artifact Panel、Artifact Handoff、Pipeline Visualization、Agent Observability。	候选案例池
Step 2	筛选 P0 案例：My Virtual Office、Cursor、Claude Code、Codex、OpenHands、Confluence PRD、GitHub PR、Allure、Claude Agent Coworking、n8n、AgentPrism。	重点案例卡片
Step 3	完成 Interface Pattern Benchmark，分别输出四类核心模式总结。	模式库
Step 4	给 mentor 三个方向：专业 SOP Dashboard、Office Map + Artifact Panel、完整 AI Company Simulation。	方向建议
Step 5	制作概念原型：Office Map、PM/Dev/QA Workspace、三种 Artifact Panel、Handoff 动效。	概念原型

附录 A：建议的 Artifact 元信息骨架

元信息	说明
Artifact Type	PRD、User Story、Code Change、Build、Test Report 等。
Owner Workspace	产出该 Artifact 的岗位空间，例如 PM Office、Dev Office、QA Lab。
Status	Draft、Reviewed、Submitted、Received、Accepted、Rejected、Blocked、Done。
Version	v1、v2、v3，支持被退回和迭代。
Submitted By / Reviewed By	提交人和审核人，支持人类和 AI Agent 共同留痕。
Consumed By	下游消费方，例如 Dev Workspace 或 QA Workspace。
Evidence	生成过程摘要、人工审核记录、关键结论、Trace 链接。
Audit Log	提交、接收、退回、澄清、版本更新的时间线。

附录 B：参考来源

[1] My Virtual Office GitHub: <https://github.com/eliautobot/my-virtual-office>
[2] My Virtual Office: <https://myvirtualoffice.ai/>
[3] WorkAdventure GitHub: <https://github.com/workadventure/workadventure>
[4] WorkAdventure Open Source: <https://workadventu.re/open-source/>
[5] Gather Virtual Office: <https://www.gather.town/virtual-office>

- [6] **AgentOffice GitHub:** <https://github.com/harishkotra/agent-office>
- [7] **Cursor:** <https://cursor.com/>
- [8] **Claude Code Subagents:** <https://code.claude.com/docs/en/sub-agents>
- [9] **OpenAI Codex Cloud:** <https://developers.openai.com/codex/cloud>
- [10] **OpenAI Code Generation Guide:** <https://developers.openai.com/api/docs/guides/code-generation>
- [11] **OpenHands:** <https://www.openhands.dev/>
- [12] **OpenHands GitHub:** <https://github.com/OpenHands/openhands>
- [13] **Replit Agent:** <https://replit.com/products/agent>
- [14] **Dify:** <https://dify.ai/>
- [15] **Confluence PRD Template:** <https://www.atlassian.com/software/confluence/templates/product-requirements>
- [16] **Linear Project Documents:** <https://linear.app/docs/project-documents>
- [17] **Linear Issue Status:** <https://linear.app/docs/configuring-workflows>
- [18] **GitHub Pull Request Docs:** <https://docs.github.com/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- [19] **GitHub Protected Branches:** <https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
- [20] **Jira Workflows:** <https://support.atlassian.com/jira-cloud-administration/docs/work-with-issue-workflows/>
- [21] **GitHub Actions Docs:** <https://docs.github.com/actions>
- [22] **Allure Report:** <https://allurereport.org/>
- [23] **Playwright Test Reporters:** <https://playwright.dev/docs/test-reporters>
- [24] **n8n:** <https://n8n.io/>
- [25] **Langflow:** <https://www.langflow.org/>
- [26] **AgentPrism GitHub:** <https://github.com/evilmartians/agent-prism>
- [27] **ClawMetry:** <https://clawmetry.com/>
- [28] **LangSmith Observability:** <https://docs.langchain.com/langsmith/observability>
- [29] **Arize Phoenix:** <https://arize.com/docs/phoenix>
- [30] **Claude Agent Coworking Workflows reference:** <https://fast.io/resources/claude-agent-coworking-workflows/>